Student Dropout Prediction

Using Machine Learning Algorithms

ECEN 429 · Final Presentation · Aishwaryaa Kannan & Ibtisam Haseeb

Martins et al. (2021). UCI ML Repository. doi:10.24432/C5MC89 · Texas A&M University at Qatar · April 2025

Introduction

Problem Statement

Student dropout is a major challenge in higher education. Traditional identification methods are reactive — advisors only discover at-risk students after grades drop, often too late. Machine learning enables early prediction from enrollment and first-semester data, allowing timely, targeted intervention.

32%

average global university dropout rate

OECD (2022)

$16B+

annual economic cost of student dropout (US)

OECD (2022)

OECD (2022). Education at a Glance. doi:10.1787/3197152b-en

Dataset Description

Source: Martins, M.V. et al. (2021). UCI ML Repository — Predict Students' Dropout and Academic Success. doi:10.24432/C5MC89

https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success [Accessed: March 2025]

4,424	36	3	4
Student records	Input features	Output classes	Input feature categories

Academic Features - 16

- Previous qualification grade
- Admission grade (0–200)
- Age at enrollment
- Daytime/evening attendance
- Units enrolled/approved/grade sem 1
- Units enrolled/approved/grade sem 2

Socioeconomic Features - 8

- Scholarship holder (0/1)
- Tuition fees up to date (0/1)
- Debtor status (0/1)
- Displaced student (0/1)
- Mother's & Father's education
- Mother's & Father's occupation

Macroeconomic Features - 3

- GDP at enrollment
- Unemployment rate
- Inflation rate

Demographic Features - 9

- Gender
- Marital Status

Output (Target Variable): **Dropout** (32.1%) · **Enrolled** (17.9%) · **Graduate** (49.9%)

Literature Review

Martins et al. (2021)

"Early Prediction of Student's Performance in Higher Education"

DSAI 2021 · LNCS Vol. 12941, pp. 166–179 · Springer

doi:10.1007/978-3-030-93859-8_16

Finding: Introduced the dataset we use. Random Forest performed best (~81% accuracy). Academic and socioeconomic features both identified as key predictors.

Villar & de Andrade (2024)

"Supervised ML for Predicting Student Dropout: A Comparative Study"

Discover Artificial Intelligence, Vol. 4, Art. 2 · Springer Nature

doi:10.1007/s44163-023-00079-z

Finding: Boosting algorithms (LightGBM, CatBoost) outperform classical ML. SMOTE applied to address class imbalance.

Kabathova & Drlik (2021)

"Towards Predicting Student Dropout Using Different ML Techniques"

Applied Sciences, Vol. 11, No. 7, p. 3130 · MDPI

doi:10.3390/app11073130

Finding: SVM and LR compared across course types. First-semester grades consistently strongest predictor.

Kruger et al. (2023)

"An Explainable ML Approach for Student Dropout Prediction"

Expert Systems with Applications, Vol. 233, Art. 120933 · Elsevier

doi:10.1016/j.eswa.2023.120933

Finding: Ensemble model (LR + NN + DT) achieved 89% accuracy. SHAP values used to explain individual predictions.

All sources accessed via Google Scholar and publisher websites · March 2025

Algorithms — Description, Input & Output

K-Nearest Neighbors

KNN

Classifies a student by finding the K most similar students in the training set and taking a majority vote among their labels.

Input: 36 student features (numerical), scaled using StandardScaler. Scaling required — KNN uses Euclidean distance and is sensitive to feature magnitude.

Output: One predicted class label: Dropout, Enrolled, or Graduate

Why chosen: Simple distance-based baseline. Requires no training phase.

Support Vector Machine

SVM

Finds the optimal hyperplane that maximally separates the three classes in high-dimensional feature space using an RBF kernel.

Input: 36 student features (numerical), scaled using StandardScaler. Scaling required for kernel convergence.

Output: Class with maximum margin from the decision hyperplane (RBF kernel, one-vs-one strategy for 3 classes)

Why chosen: Effective in high-dimensional spaces. Strong on structured tabular data.

Logistic Regression

LR

Estimates class probabilities using a linear combination of all 36 features, passed through a softmax function (multinomial). Predicts the class with highest probability.

Input: 36 student features (numerical), scaled using StandardScaler. Model outputs a probability score for each of the 3 classes.

Output: Probability per class via softmax → argmax → predicted class label

Why chosen: Most interpretable model — feature coefficients show which inputs drive dropout.

Random Forest

RF

Ensemble of 100s of decision trees, each trained on a random bootstrap sample with a random feature subset. Final prediction by majority vote.

Input: 36 student features (numerical). No scaling required — tree-based splits use rank order, not magnitude.

Output: Majority vote across 100 trees → predicted class + feature importance scores per feature

Why chosen: Typically strongest on tabular data. Feature importance scores enable our novelty analysis.

Methodology Overview

1

Data & Preprocessing

4,424 students · 36 features · StandardScaler · 80/20 stratified split

What we work with and how we prepare it

2

Hyperparameter Tuning

GridSearchCV with 5-fold stratified cross-validation per model

Finding the best settings for each algorithm automatically

3

Evaluation Metrics

Accuracy · Precision · Recall · F1-macro · AUC

How we measure and compare model performance

4

Our Experiments

Best model comparison · Feature group analysis · SHAP analysis · VIF screening

Four research questions answered through four experiments

Data Preprocessing — Train / Test Split

80 / 20 Stratified Split — 4,424 students total

Training Set — 3,539 students (80%)

Test Set
885 (20%)

Stratified — class proportions preserved + Feature Scaling using StandardScaler

Class distribution in both sets:

Dropout	Enrolled	Graduate
32.1%	17.9%	49.9%
~1,421 students	~794 students	~2,209 students

5-Fold Cross-Validation (on training set only)

Validate	Train	Train	Train	Train	Fold 1
Train	Validate	Train	Train	Train	Fold 2
Train	Train	Validate	Train	Train	Fold 3
Train	Train	Train	Validate	Train	Fold 4
Train	Train	Train	Train	Validate	Fold 5

Each fold gets a turn as the validation set. Average of 5 scores = final CV score.

random_state = 42 · Scikit-learn train_test_split with stratify=y

Hyperparameter Tuning — GridSearchCV

Tests every combination of hyperparameter values using 5-fold cross-validation — splits training data into 5 parts, trains on 4, validates on 1, repeats 5 times, picks the combination with the best average F1-macro.

```python
print(" Hyperparameter Tuning (GridSearchCV, 5-fold CV)")

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

param_grids = {
    'KNN': {'n_neighbors': [3, 5, 7, 11, 15]},
    'SVM': {'C': [0.1, 1, 10], 'gamma': ['scale', 'auto']},
    'LR' : {'C': [0.01, 0.1, 1, 10]},
    'RF' : {'n_estimators': [50, 100, 200], 'max_depth': [None, 10, 20]},
}

base_estimators = {
    'KNN': KNeighborsClassifier(),
    'SVM': SVC(kernel='rbf', probability=True, random_state=RANDOM_STATE),
    'LR' : LogisticRegression(multi_class='multinomial', max_iter=1000,
                              random_state=RANDOM_STATE),
    'RF' : RandomForestClassifier(random_state=RANDOM_STATE),
}

best_params  = {}
best_models  = {}
cv_scores    = {}
for name, estimator in base_estimators.items():
    gs = GridSearchCV(estimator, param_grids[name],
                      cv=cv, scoring='f1_macro', n_jobs=-1, refit=True)
    gs.fit(X_train_sc, y_train)
    best_params[name] = gs.best_params_
    best_models[name] = gs.best_estimator_
    cv_scores[name]   = round(gs.best_score_, 4)
    print(f"  {name:4s}  Best params : {gs.best_params_}  "
          f"| Best CV F1 : {gs.best_score_:.4f}")

MODEL_NAMES = list(best_models.keys())
```

GridSearchCV

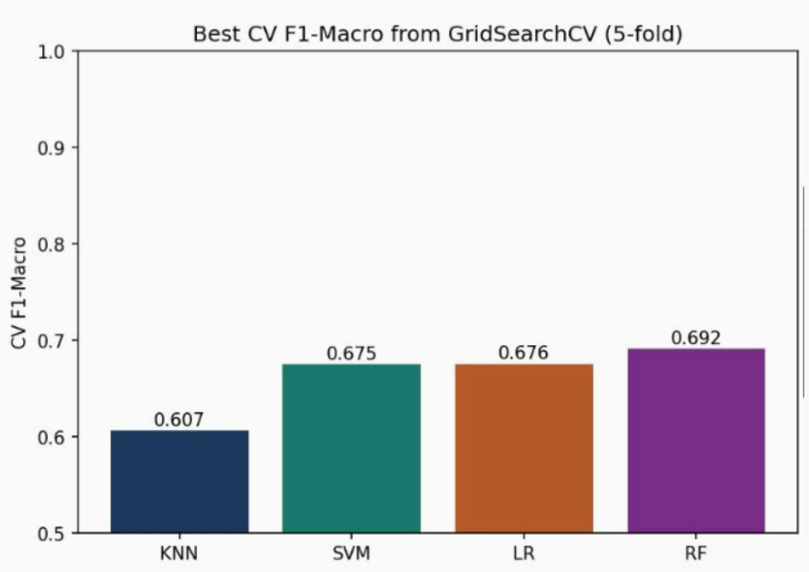
Tries every combination of hyperparameter values:

KNN:	n_neighbors: 3,5,7,11,15
SVM:	C: 0.1,1,10 · gamma: scale,auto
LR:	C: 0.01,0.1,1,10
RF:	n_estimators: 50,100,200 max_depth: None,10,20

Scoring: F1-macro. Best combo retrained on full training set.

Scoring metric: F1-macro · 5-fold stratified cross-validation · Best combination retrained on full training set · random_state=42

Hyperparameter Tuning — GridSearchCV (5-Fold CV)



Best Parameters Found

KNN	n_neighbors: 5	**0.607**
SVM	C: 10, gamma: scale	**0.675**
LR	C: 10	**0.676**
RF	n_estimators: 200 max_depth: 20	**0.692**

Output:

```
Hyperparameter Tuning (GridSearchCV, 5-fold CV)
 KNN   Best params : {'n_neighbors': 5}  | Best CV F1 : 0.6069
 SVM   Best params : {'C': 10, 'gamma': 'scale'}  | Best CV F1 : 0.6749
 LR    Best params : {'C': 10}  | Best CV F1 : 0.6757
 RF    Best params : {'max_depth': 20, 'n_estimators': 200}  | Best CV F1 : 0.6917
```

Key Finding

RF ranked highest during cross-validation (CV F1=0.692). SVM and LR performed similarly (0.675, 0.676). KNN lagged behind as expected — simple distance-based models struggle with 36 features.

All models retrained on full training set after tuning · Scores shown are CV scores — test set results on next slide

RQ1 — Model Comparison: Full Feature Set (36 Features)

```python
def eval_model(model, Xtr, Xte, ytr, yte):
    model.fit(Xtr, ytr)
    yp = model.predict(Xte)
    return {
        'Accuracy' : round(accuracy_score(yte, yp), 4),
        'Precision': round(precision_score(yte, yp, average='macro', zero_division=0), 4),
        'Recall'   : round(recall_score(yte, yp,    average='macro', zero_division=0), 4),
        'F1-Macro' : round(f1_score(yte, yp,        average='macro', zero_division=0), 4),
    }, yp

results_full = {}
preds_full   = {}

for name, model in best_models.items():
    met, yp = eval_model(model, X_train_sc, X_test_sc, y_train, y_test)
    results_full[name] = met
    preds_full[name]   = yp
    print(f"  {name:4s}  CV F1={cv_scores[name]}  ->  "
          f"Test: Acc={met['Accuracy']}  P={met['Precision']}  "
          f"R={met['Recall']}  F1={met['F1-Macro']}")

df_full   = pd.DataFrame(results_full).T
best_name = df_full['F1-Macro'].idxmax()
print(f"\n  >>> Best model : {best_name}  (Test F1 = {df_full.loc[best_name,'F1-Macro']})")
```

Test Set Performance

Model	Accuracy	F1-macro	AUC
KNN	0.668	0.573	0.782
SVM	0.719	0.643	0.841
LR	0.772	0.688	0.870
RF ★	**0.773**	**0.696**	**0.884**

RF is best

F1=0.696, AUC=0.884 — highest across all metrics

LR is competitive

F1=0.688, AUC=0.870 — nearly matches RF. More interpretable for real deployment

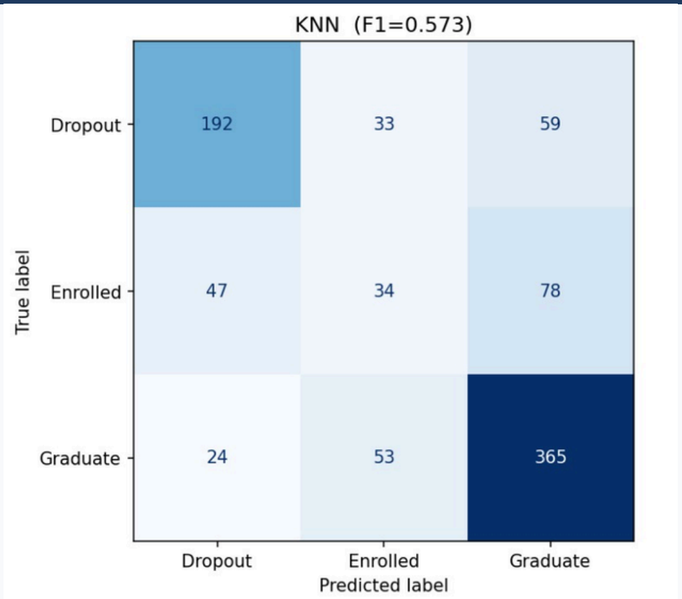
KNN worst

F1=0.573 — high dimensionality across 36 features

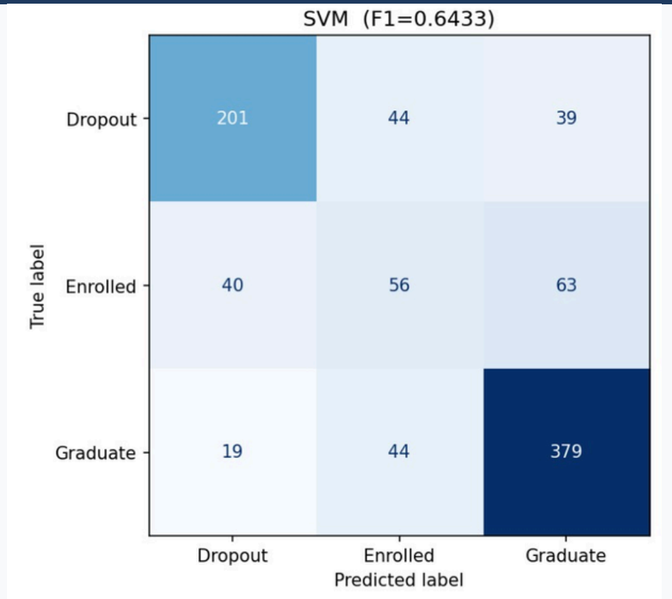
Sokolova & Lapalme (2009). Information Processing & Management, Vol. 45, No. 4, pp. 427–437. doi:10.1016/j.ipm.2009.03.002

Confusion Matrices — All Four Models

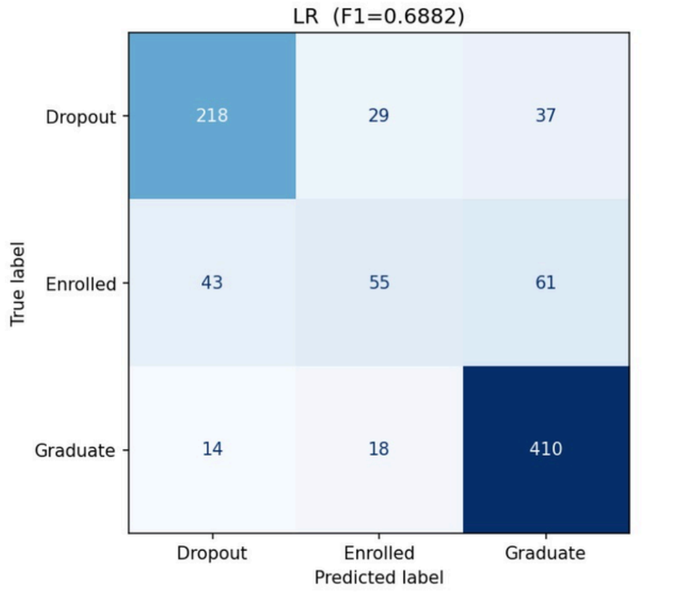
KNN (F1=0.573)



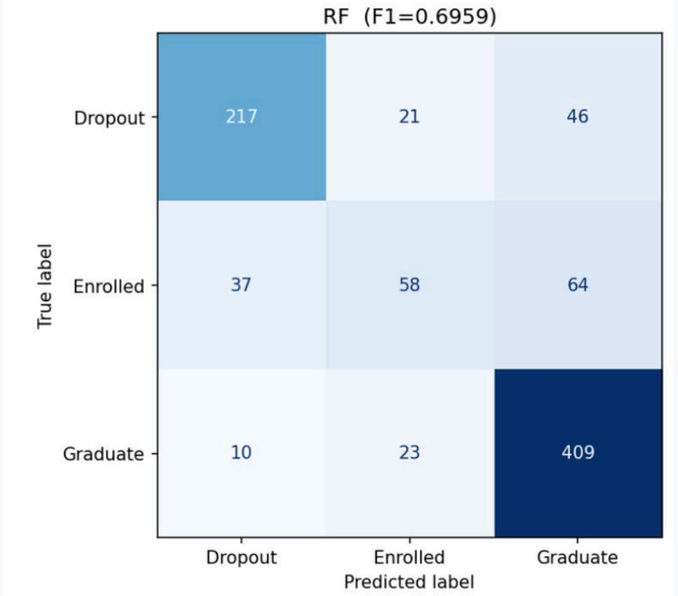
SVM (F1=0.643)



LR (F1=0.688)



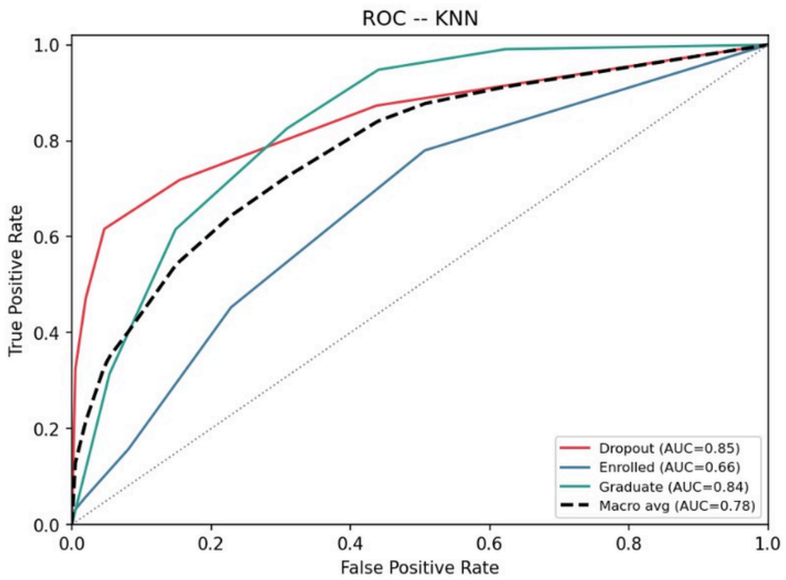
RF (F1=0.696)



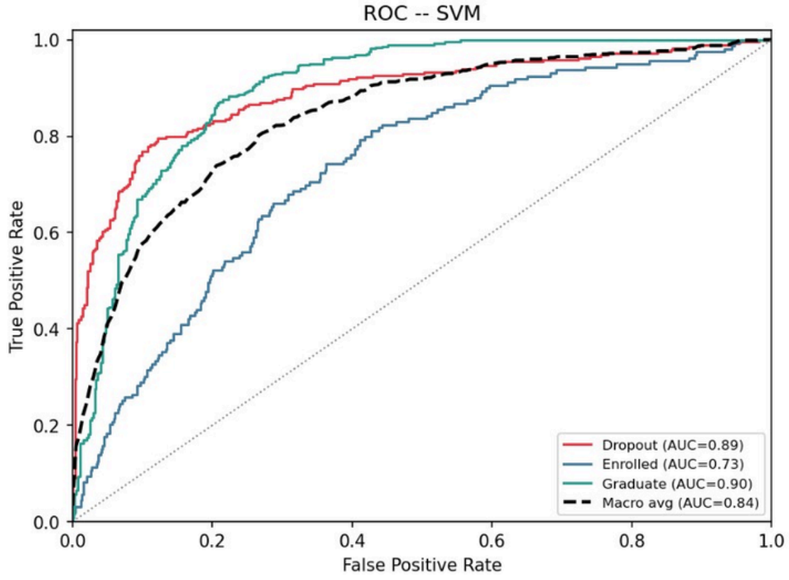
Key pattern: Enrolled recall never exceeds 36% across all models — enrolled students look similar to future graduates, making them genuinely hard to classify.

ROC Curves — One-vs-Rest per Class

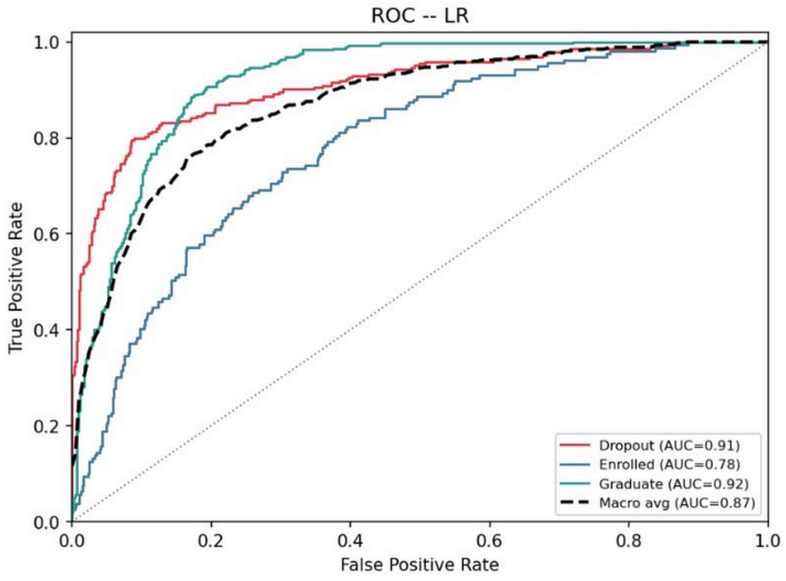
KNN · Macro AUC = 0.782



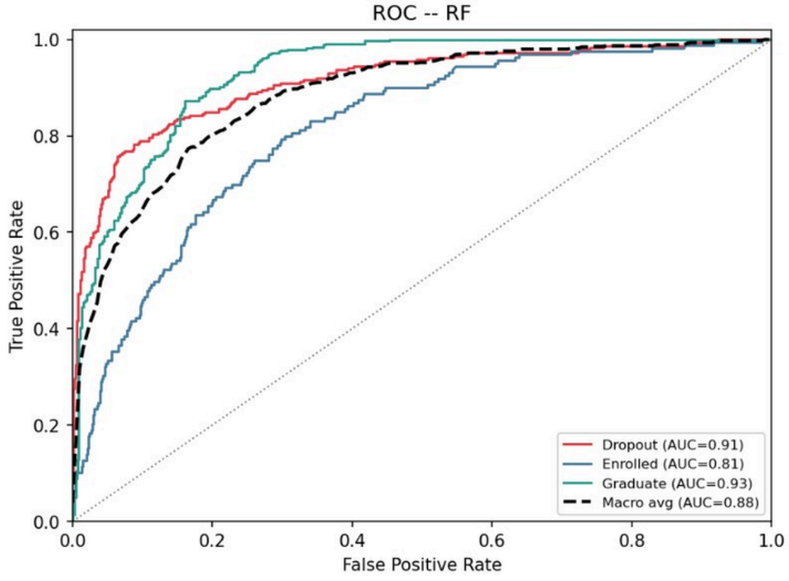
SVM · Macro AUC = 0.841



LR · Macro AUC = 0.870



RF · Macro AUC = 0.884



AUC tells a more optimistic story — RF Macro AUC=0.884 shows strong discriminative ability. Even Enrolled AUC=0.812 shows the model can rank them correctly, but struggles with clean binary decisions.

RQ2 — Feature Group Isolation

```python
feature_groups = {
    'Academic'      : academic_cols,
    'Socioeconomic' : socio_cols,
    'Demographic'   : demographic_cols,
    'Macroeconomic' : macro_cols,
    'All Features'  : list(X_raw.columns),
}

rq2_rows = []
for grp, cols in feature_groups.items():
    if not cols:
        continue
    sc_g  = StandardScaler()
    Xg_tr = sc_g.fit_transform(X_train[cols])
    Xg_te = sc_g.transform(X_test[cols])
    row   = {'Group': grp, 'N Features': len(cols)}
    for name in MODEL_NAMES:
        m = type(best_models[name])(**best_models[name].get_params())
        met, _ = eval_model(m, Xg_tr, Xg_te, y_train, y_test)
        row[f'{name} F1'] = met['F1-Macro']
    rq2_rows.append(row)
    print(f"  {grp:15s} ({len(cols):2d} feat): "
          + "  ".join(f"{n}={row[f'{n} F1']}" for n in MODEL_NAMES))

df_rq2 = pd.DataFrame(rq2_rows).set_index('Group')
```

F1-Macro by Group

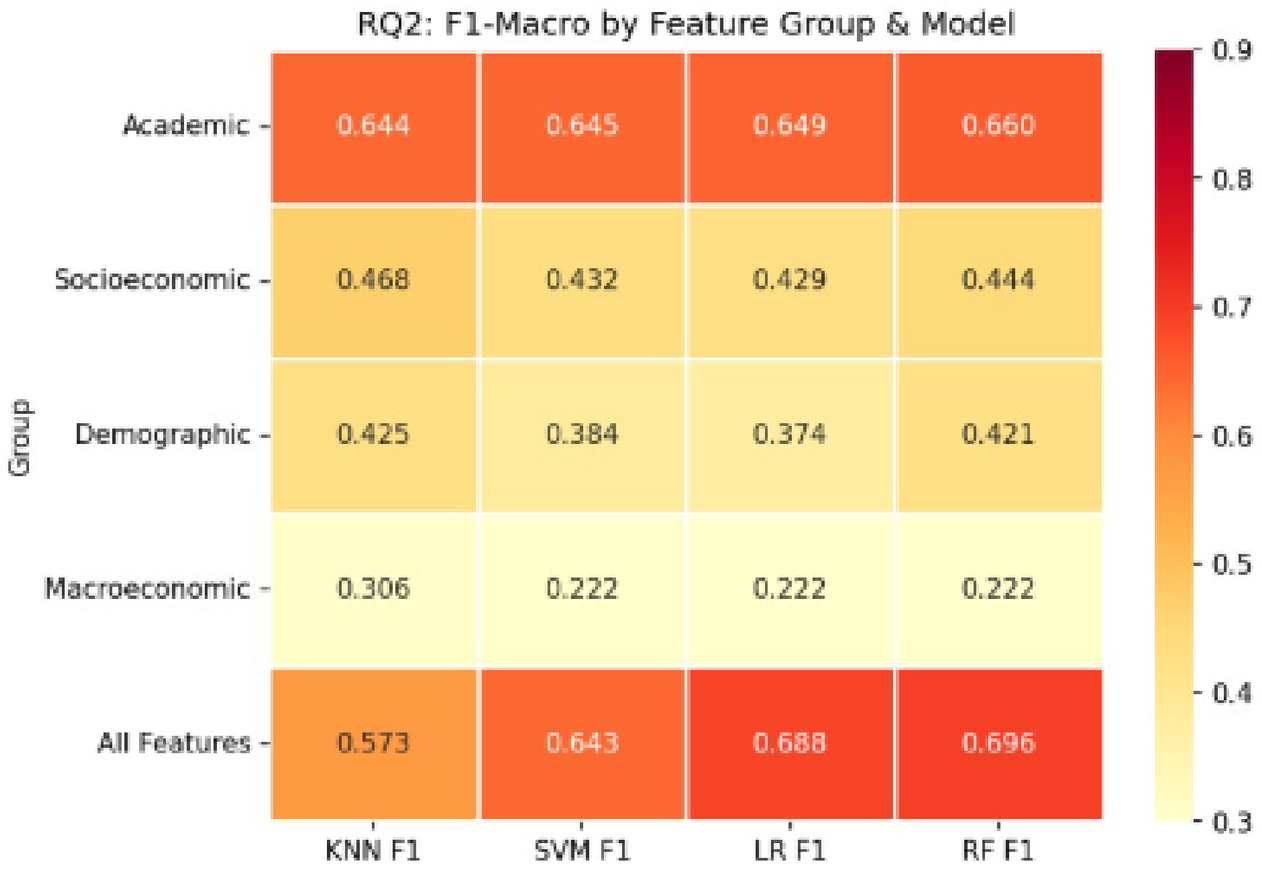
Group (#feat)	RF F1	Note
Academic (16)	**0.660**	*RF best*
Socioeconomic (8)	**0.444**	*Early warning*
Demographic (9)	**0.421**	*Marginal value*
Macroeconomic (3)	**0.222**	*~Random*
All Features (36)	**0.696**	*Baseline*

Academic alone → 95% of full model: RF with 16 academic features achieves F1=0.660 vs 0.696 with all 36

Socioeconomic → useful at enrollment: F1=0.444 but available BEFORE any grades — day-one early warning

KNN anomaly: KNN F1 IMPROVES with academic-only (0.644 vs 0.573) — fewer features reduces distance noise

RQ2 — Feature Group Isolation

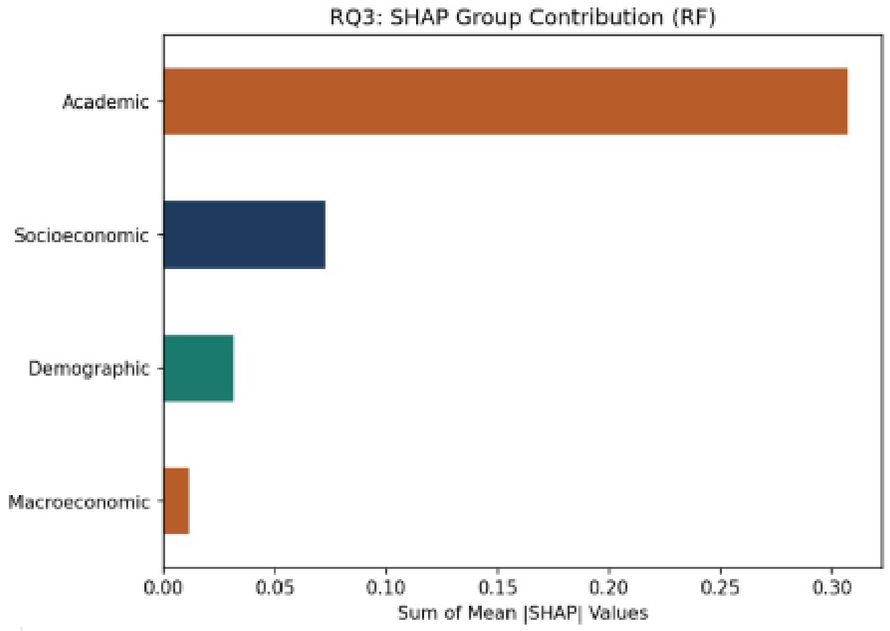


F1-Macro by Group

Group (#feat) RF	F1	Note
Academic (16)	**0.660**	*RF best*
Socioeconomic (8)	**0.444**	*Early warning*
Demographic (9)	**0.421**	*Marginal value*
Macroeconomic (3)	**0.222**	*~Random*
All Features (36)	**0.696**	*Baseline*

RQ3 — SHAP Feature Importance (Random Forest)

```python
best_fitted = best_models[best_name]
X_test_df   = pd.DataFrame(X_test_sc, columns=X_raw.columns)
X_sample    = X_test_df.sample(min(200, len(X_test_df)), random_state=RANDOM_STATE)

if best_name == 'RF':
    explainer = shap.TreeExplainer(best_fitted)
    shap_vals = explainer.shap_values(X_sample)
    shap_abs  = np.mean([np.abs(sv) for sv in shap_vals], axis=0)
else:
    bg        = shap.sample(X_sample, 50)
    explainer = shap.KernelExplainer(best_fitted.predict_proba, bg)
    shap_vals = explainer.shap_values(X_sample)
    shap_abs  = np.mean([np.abs(sv) for sv in shap_vals], axis=0)

shap_mean = pd.Series(shap_abs.mean(axis=1), index=X_raw.columns)

def grp_shap(cols):
    return shap_mean[[c for c in cols if c in shap_mean.index]].sum()

shap_groups = {
    'Academic'      : grp_shap(academic_cols),
    'Socioeconomic' : grp_shap(socio_cols),
    'Demographic'   : grp_shap(demographic_cols),
    'Macroeconomic' : grp_shap(macro_cols),
}
print("  Summed mean |SHAP| per group:")
for g, s in sorted(shap_groups.items(), key=lambda x: -x[1]):
    print(f"    {g:15s}: {s:.4f}")
```

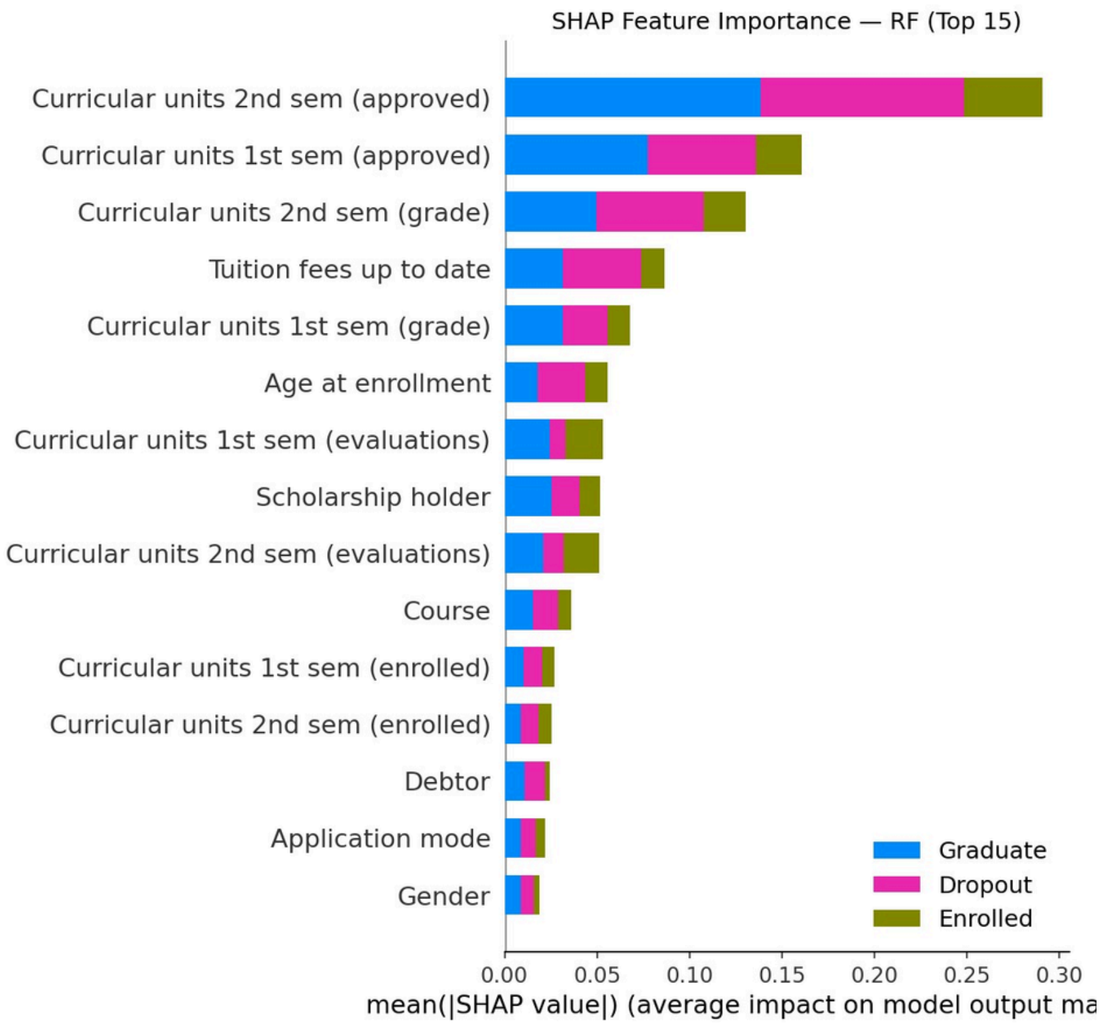


Group Contribution (Sum |SHAP|)

Academic	0.307	1st
Socioeconomic	0.073	2nd
Demographic	0.037	3rd
Macroeconomic	0.012	4th

Academic features contribute 4× more than socioeconomic (0.307 vs 0.073). Top 3 features: units approved sem 2, units approved sem 1, grade sem 2 — all academic. First non-academic: Tuition fees up to date (4th).

RQ3 — SHAP Feature Importance (Random Forest)



Academic features contribute 4× more than socioeconomic (0.307 vs 0.073). Top 3 features: units approved sem 2, units approved sem 1, grade sem 2 — all academic. First non-academic: Tuition fees up to date (4th).

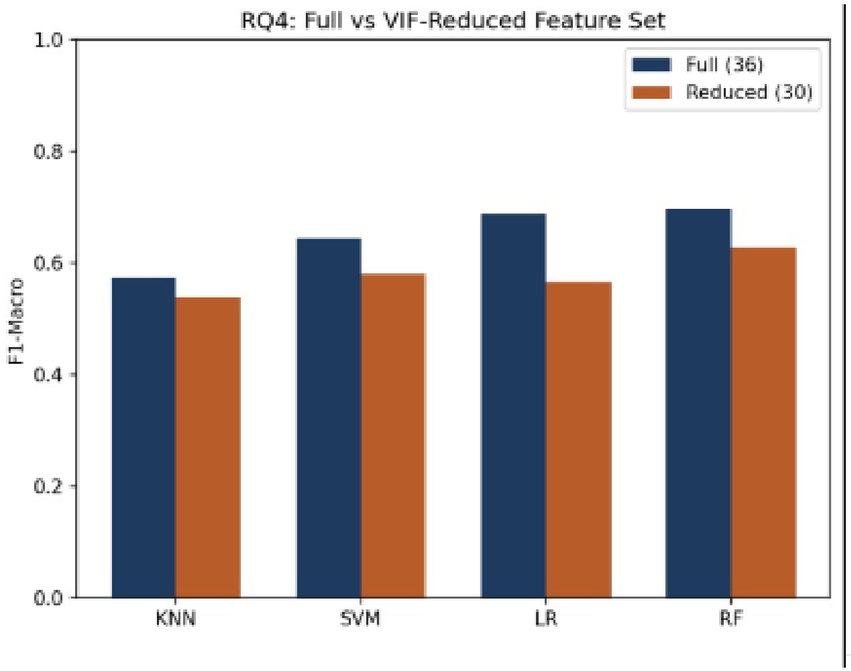
RQ4 — VIF Screening: Does Removing Redundant Features Help?

```python
vif_df = pd.DataFrame({
    'feature': X_raw.columns,
    'VIF'    : [variance_inflation_factor(X_train_sc, i)
                for i in range(X_train_sc.shape[1])]
}).sort_values('VIF', ascending=False)

print("\n  Top 10 VIF scores:")
print(vif_df.head(10).to_string(index=False))

high_vif     = vif_df[vif_df['VIF'] > 10]['feature'].tolist()
reduced_cols = [c for c in X_raw.columns if c not in high_vif]
X_train_red  = X_train_sc_df[reduced_cols].values
X_test_red   = X_test_sc_df[reduced_cols].values
print(f"\n  Removed (VIF > 10) : {len(high_vif)}  ->  {high_vif}")
print(f"  Remaining          : {len(reduced_cols)} features")

print("\n  Retraining all models on reduced feature set...")
results_red = {}
for name, model in best_models.items():
    m2 = type(model)(**model.get_params())
    met, _ = eval_model(m2, X_train_red, X_test_red, y_train, y_test)
    results_red[name] = met

df_red  = pd.DataFrame(results_red).T
delta   = df_red['F1-Macro'] - df_full['F1-Macro']
comp_df = pd.DataFrame({
    'Full (36)'    : df_full['F1-Macro'],
    'Reduced (30)': df_red['F1-Macro'],
    'Delta F1'     : delta
})
print("\n  RQ4 — Full vs Reduced Comparison:")
print(comp_df.to_string())
```

Features Removed (VIF > 10)

Units 1st sem (enrolled)	**23.77**
Units 2nd sem (enrolled)	**16.99**
Units 1st sem (credited)	**16.79**
Units 1st sem (approved)	**13.35**
Units 2nd sem (credited)	**13.29**
Units 2nd sem (approved)	**10.95**

RQ4 — VIF Screening: Does Removing Redundant Features Help?



Impact of Removing 6 Features (36 → 30)

Model	Full (36)	Reduced (30)	Delta F1
KNN	0.573	0.540	**-0.033**
SVM	0.643	0.581	**-0.063**
LR	0.688	0.565	**-0.123**
RF	0.696	0.628	**-0.068**

Counter-intuitive finding:

All models DROPPED in performance after removing the 6 flagged features. VIF > 10 does not always mean remove. Villar & de Andrade (2024) were right to use all 36 features — this experiment provides the evidence they were missing.

Conclusions

RQ1

Random Forest is best — F1=0.696, AUC=0.884

LR is competitive (F1=0.688) and more interpretable. KNN worst at 0.573 due to curse of dimensionality.

RQ2

Academic features alone achieve 95% of full model performance

RF academic-only F1=0.660 vs 0.696 full. Socioeconomic (F1=0.444) is useful for early warning before grades exist.

RQ3

Academic features contribute 4× more than socioeconomic (SHAP: 0.307 vs 0.073)

Top 3 features all academic. First non-academic: Tuition fees (4th). Socioeconomic is real but secondary.

RQ4

Removing multicollinear features HURTS performance (F1 drops 0.03–0.12)

VIF > 10 does not mean remove. Correlated curricular features encode collective signal no other features replace.

Limit

Single institution (Portugal, 2008–2019) · Enrolled class genuinely ambiguous

Enrolled recall never exceeds 36% — these students look like future graduates.

Future Directions

Building on the findings of this project — three directions for future research and application

Early Warning System Deployment

Build a web dashboard where advisors input student data and receive a dropout risk score with a SHAP-based explanation of which features drove the prediction. Directly connects all four RQs to real-world use.

Per-Group Hyperparameter Tuning

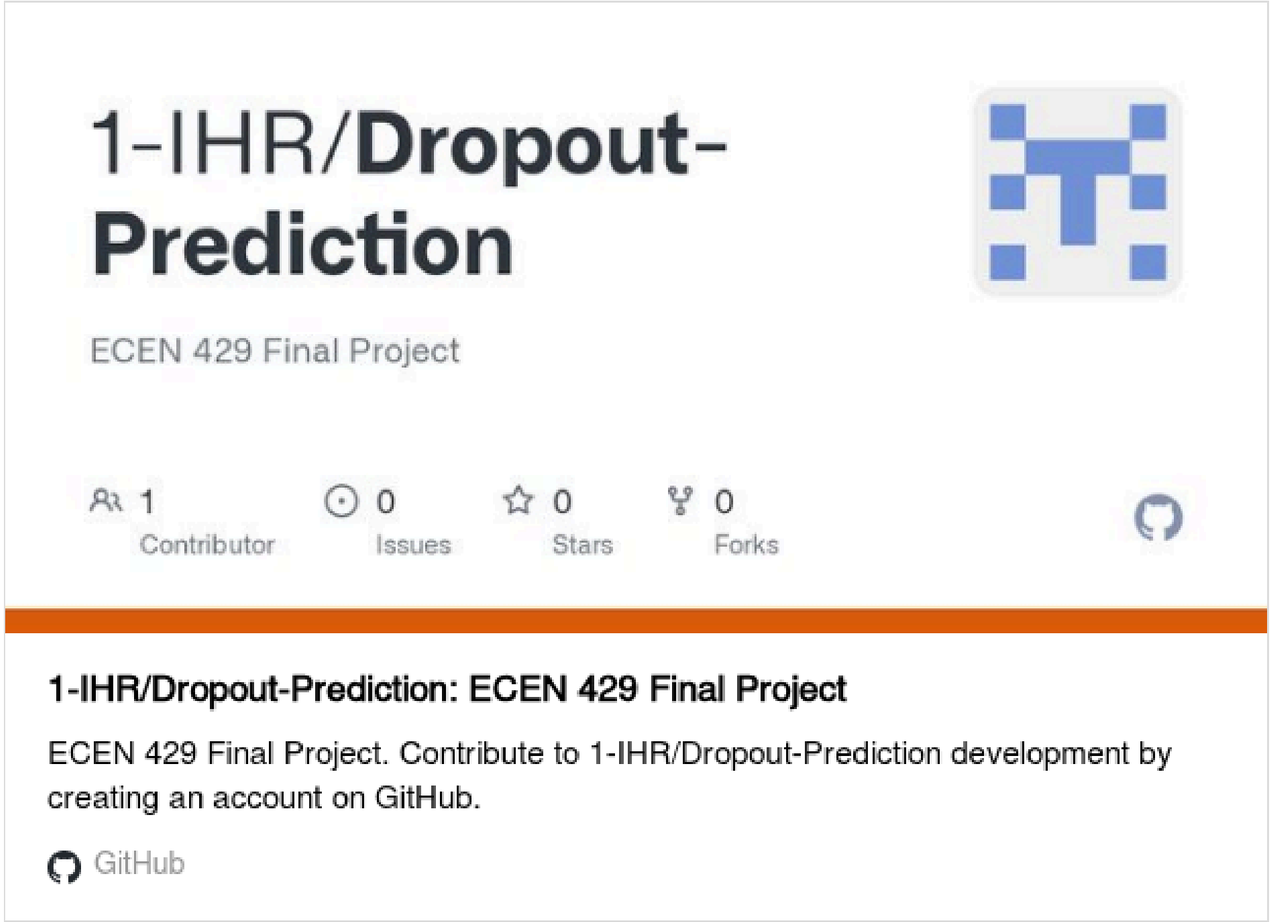
In our RQ2 experiments, hyperparameters were tuned on all 36 features and reused per group. Tuning separately per feature group may yield improved performance for each isolated experiment.

Early Prediction Timing Experiment

Test model accuracy using only enrollment-time data (before semester 1), then add semester 1 data, then semester 2. Quantifies exactly how early a reliable prediction can be made.

Villar & de Andrade (2024). Discover Artificial Intelligence. doi:10.1007/s44163-023-00079-z · Martins et al. (2021). doi:10.24432/C5MC89

Github Repo



Contains code + dataset + all figures

Villar & de Andrade (2024). Discover Artificial Intelligence. doi:10.1007/s44163-023-00079-z · Martins et al. (2021). doi:10.24432/C5MC89

References

1. Martins, M.V. et al. (2021). "Early Prediction of Student's Performance in Higher Education." DSAI 2021, LNCS Vol. 12941, pp. 166–179. Springer. doi:10.1007/978-3-030-93859-8_16 | Dataset: doi:10.24432/C5MC89 | https://archive.ics.uci.edu/dataset/697 [Accessed: March 2025]

2. Villar, A. & de Andrade, C.R.V. (2024). "Supervised ML for Predicting Student Dropout: A Comparative Study." Discover Artificial Intelligence, Vol. 4, Art. 2. Springer Nature. doi:10.1007/s44163-023-00079-z [Accessed: March 2025]

3. Kabathova, J. & Drlik, M. (2021). "Towards Predicting Student's Dropout Using Different ML Techniques." Applied Sciences, Vol. 11, No. 7, p. 3130. MDPI. doi:10.3390/app11073130 [Accessed: March 2025]

4. Krüger, J., Britto, A. & Barddal, J. (2023). "An Explainable ML Approach for Student Dropout Prediction." Expert Systems with Applications, Vol. 233, Art. 120933. Elsevier. doi:10.1016/j.eswa.2023.120933

5. Prenkaj, B. et al. (2020). "A survey of ML approaches for student dropout prediction in online courses." ACM Computing Surveys, Vol. 53, No. 3, pp. 1–34. doi:10.1145/3388440

6. Frontiers in Education (2025). "Analyzing dropout and predicting academic performance using AI." Section: Higher Education. doi:10.3389/feduc.2025.1698505 [Accessed: March 2025]

7. Sokolova, M. & Lapalme, G. (2009). "A systematic analysis of performance measures for classification tasks." Information Processing & Management, Vol. 45, No. 4, pp. 427–437. doi:10.1016/j.ipm.2009.03.002

8. OECD (2022). Education at a Glance 2022: OECD Indicators. OECD Publishing, Paris. doi:10.1787/3197152b-en [Accessed: March 2025]